

A method for forecasting defect backlog in large streamline software development projects and its industrial evaluation

Mirosław Staron^{a,*}, Wilhelm Meding^b, Bo Söderqvist^b

^a Department of Applied IT, Chalmers and University of Gothenburg, SE 412-96 Gothenburg, Sweden

^b Ericsson SW Research, Ericsson AB, Sweden

ARTICLE INFO

Article history:

Received 18 August 2009

Received in revised form 6 May 2010

Accepted 10 May 2010

Available online 31 May 2010

Keywords:

Quality metrics

Defect prediction

Early warning

LEAN software development

Streamline development

ABSTRACT

Context: Predicting a number of defects to be resolved in large software projects (*defect backlog*) usually requires complex statistical methods and thus is hard to use on a daily basis by practitioners in industry. Making predictions in simpler and more robust way is often required by practitioners in software engineering industry.

Objective: The objective of this paper is to present a simple and reliable method for forecasting the level of defect backlog in large, lean-based software development projects.

Method: The new method was created as part of an action research project conducted at Ericsson. In order to create the method we have evaluated multivariate linear regression, expert estimations and analogy-based predictions w.r.t. their accuracy and ease-of-use in industry. We have also evaluated the new method in a life project at one of the units of Ericsson during a period of 21 weeks (from the beginning of the project until the release of the product).

Results: The method for forecasting the level of defect backlog uses an indicator of the trend (an arrow) as a basis to forecast the level of defect backlog. Forecasts are based on moving average which combined with the current level of defect backlog was found to be the best prediction method (Mean Magnitude of Relative Error of 16%) for the level of future defect backlog.

Conclusion: We have found that ease-of-use and accuracy are the main aspects for practitioners who use predictions in their work. In this paper it is concluded that using the simple moving average provides a sufficiently-good accuracy (much appreciated by practitioners involved in the study). We also conclude that using the indicator (forecasting the trend) instead of the absolute number of defects in the backlog increases the confidence in our method compared to our previous attempts (regression, analogy-based, and expert estimates).

© 2010 Elsevier B.V. All rights reserved.

1. Introduction

Market driven software engineering increases the demands on companies to be more market-oriented and customer responsive. Companies are facing demands to deliver customized products in shorter time frames; a situation which is not handled very well by the traditional processes [1,2]. For small and medium sized projects, Agile methods [3] like Scrum [4] or eXtreme Programming [5] are often used. For hardware oriented companies processes based on Lean [6,7] principles are often applied. In large software companies like Ericsson a mix of these two is needed as neither Lean nor Agile are directly applicable. Ericsson has developed and applied such a mix – Streamline Development (SD [8]) which promises short development times and increased market agility when developing large and complex software systems. SD is based

on the idea that software development in large projects can be done in Lean-like style, where the metaphor of factory-like assembly stations is used for software development. Defect prediction is very important in this environment because software builds are based on previous releases and therefore the quality of the releases has to be constantly very high. This implies that all defects have to be addressed as soon as possible after their discovery, which requires resources, which might be scarce during certain phases of project. The removal of the defects, if not properly planned, might lead to delays (inability to deliver the software of desired quality) of increased costs of the project (e.g. overtime, additional resources). Predicting the defect backlog becomes very important in this situation. In this context the *defect backlog* is the set of all known defects which remain to be resolved in the project.

The main contribution of this paper is a new method for predicting the number of defects in the backlog in large software projects executed according to the principles of SD. The method is a result of 2 years research project conducted at Ericsson AB when

* Corresponding author. Tel.: +46 31 772 1081.

E-mail addresses: mirosław.staron@ituniv.se (M. Staron), wilhelm.meding@ericsson.com (W. Meding), bo.soderqvist@ericsson.com (B. Söderqvist).

we found that predicting defect discovery should be replaced by predicting the level of defect backlog and communicating the trend in the defect backlog to the stakeholder (increasing or decreasing defect backlog) using a simple indicator – an arrow in our case. We refer to that as forecasting of the trend of defect backlog.

We have also found that practitioners in industry seek the credibility and reliability in the predictions and are willing to sacrifice the accuracy of the predictions for the ease-of-use or the possibility of back-tracking the predicted numbers. This observation and the close work with practitioners had led us to changing the aim of our project from providing accurate predictions of the number of defects to forecasting trends (up-constant-down). Naturally we use predictions of the number of defects when calculating trends, but then we only present this information to support the forecasts of the trends – i.e. if the trend is up then the natural question is “how much”.

As a result of our research we developed a method centered around an *indicator* of future trend of the defect backlog (an indicator is a metric with associated decision criteria, see [9,10]) based on predictions of the level of defect backlog. This indicator was much more important for the project management than the actual number of defects which was contradictory to the assumption of existing prediction models: that the number is of primary importance. Building the indicator of defect backlog trend reflects the requirements from industry that the indicator should help the project to achieve the vision of null defects at the release date. The goal of building this indicator was to improve the notification about defect inflow and the objective was to provide the stakeholder with an ISO/IEC 15939:2007 compatible indicator that will attract the attention of the stakeholder to problems with the defect backlog. Therefore our research question was:

How to forecast the level of defect backlog in Streamline Software Development projects in a straightforward way?

One of the two key concepts in our research questions was ‘straightforward’ which meant that the stakeholders would understand the formulas behind the forecast and trust these formulas [11]. The other important concept was an explicit requirement from Ericsson that it should be ‘easy to backtrack’ which meant that the stakeholder should at all times understand why the forecast show a decreasing or increasing trend.

We evaluated our method and in particular how the indicator influences the project in one of the large projects at Ericsson. The results show that the indicator indeed helps the project to decrease the number of defects in the backlog by making the project take preventive actions and that the simple prediction formula had the average prediction error of 16%.

The remaining of the paper is structured as follows. Section 2 presents the most related work to our research. Section 3 presents the context of our work – defect management at Ericsson, our research method and inadequacy of existing prediction methods in this context from our previous works. Section 4 presents an example illustrating the need for the new defect backlog indicator and requirements for the underlying forecasting model. Section 5 presents the newly developed defect backlog indicator which fulfills requirements of ease-of-use and possibility of back-tracking of causes from our industrial partner. Section 6 presents the evaluation of the indicator at Ericsson and Section 7 discusses the threats to validity of our study. Finally, Section 8 contains the conclusions and further work.

2. Related work

A substantial body of knowledge exists in predicting defect density of components or the total number of defects per component, e.g. [12–18]. Methods for predicting defect density or the number

of defects per component usually: (i) use structural metrics like component size or complexity as predictor variables and (ii) predict the total number of defects at the point of time when the component is ready to be released. A significant difference between our approach and those is the fact that we use the historical level of defect backlog as the predictor variable in our forecast model and we predict the *number of defects on a weekly basis* for a project, not when a single component is ready. In the case of lean software development, nevertheless, using complexity metrics as predictors seems not feasible, because the information about how the components are to be affected by the project is not available at the time of developing predictions; in particular the change of size and complexity is not available (which are necessary to apply models like [19]). Furthermore, for large software projects, the predictions of defect densities have been found to be insufficient [18].

Although the prediction of the level of defect backlog seems to be closely related to the area of reliability modeling, our approach differs significantly from it. In particular, reliability modeling is concerned about software reliability after release, e.g. [20–22], or [23], while our research is focused on pre-release defects. We use these pre-release defects as one of the factors affecting the workload in the project unlike reliability modeling techniques, which use defects as a measure of quality.

In early stages of our research (see [24]) we applied the Rayleigh model [25], and discovered that the defect discovery profile in the studied organization is significantly different from the profile described by the Rayleigh model. Li et al. [26] have shown that in the context of post-release defects the naïve models like moving average are not sufficient, but Weibull and Rayleigh models render more accurate results. In the light of this we recommend to use the results of Li et al. when predicting post-release defect inflow and our results/forecasting model when working with pre-release defects.

In the course of our research we encountered a number of issues related to quality of defect reports and possibility of predicting the severity of defects during submission. These aspects are not considered in our research due to the fact that they were not identified as important by the practitioners. However, we plan to conduct a follow-up case study on the possibility of using SEVERIS for predicting the severity of defects [27] and their quality [28] in order to investigate if this is possible to make more accurate and further in the future (not only 1 week) predictions of defect outflow, which is important for the predictions.

Although a part of our research is related to the issues of introducing metrics into organizations, we do not explicitly consider that question in our work. However, we followed the guidelines from such authors like Clark [29], Kilpi [30] Dekkers and McQuaid-for [31], Pfleeger et al. [32] and Bröckers et al. [33], who describe how to/why introduce measurement systems into an organization, and reflect on problems/solutions that measurements can result in.

We deliberately chose the ISO/IEC 15939 (:2002 in the start of the study and :2007 when the study concluded) in our work as the main standard when developing the indicator for the defect backlog. The reason was Ericsson’s customer orientation and thus a requirement to follow standards. A good comparison of other related measurement frameworks and models was conducted by Chirinos et al. [34], who considered such frameworks and models as ISO/IEC 15939 and Goal–Question–Metric (GQM, [35]). Despite the recommendations from Chirinos et al. to use their measurement framework (MOSME), the ISO/IEC standard was more applicable due to its wider adoption in industry, the fact that it is standardized by an international standardization body and the easy coupling of theory and practice. The use of standardized view on measurement processes also provided the possibility for future benchmarking with other organizations (e.g. as indicated in [36]).

3. Defect management at Ericsson

3.1. Research method – action research

We followed the principles of action research in our research project [37,38]. Action research is characterized by the fact that research is embedded in normal activities performed by an organization or an individual. In our case our actions were embedded in the operations of one of the units of Ericsson, employing a few hundred developers and with several large projects ongoing.¹

Action research is usually conducted in so-called cycles, which are often characterized as:

- **Action planning:** recognizing the nature of the problem in its natural environment. In our case we needed to investigate the changed reality of Lean/Streamline software development in order to understand the limitations of the previous approaches and statistical methods (as described in Section 3.4). We used the knowledge from our previous research projects and the dialog with practitioners to find these limitations and understand them. The result of the action planning is the motivating example and the requirements from practitioners.
- **Execution:** acting upon the identified problem and improving the practices. In our case we needed to develop new methods alongside with the practitioners based on the new reality of Lean/Streamline Development. The result of action execution is the new indicator for defect backlog level.
- **Evaluation:** evaluating the action in the real setting. In our case we have introduced the indicator into a large software project at Ericsson and observed how it is used on a weekly basis. The result of the evaluation is that it is sufficiently easy to use and the accuracy was satisfactory for the practitioners. As it is later stated, the indicator helped the practitioners to improve their software and thus saved considerable amount of resources.

Each of the cycles results in improvements of the current practice and each of them is intended to provide the basis for further improvements. In this paper Sections 3.3 and 3.4 summarize the first two cycles of action planning, executing, and evaluation as advocated by [37,38]. Section 4 corresponds to action planning for the final cycle of our research project.

3.2. Background – organizational context

The organization and the project within Ericsson, which we worked closely with, develop large products for the mobile telephony network. The size of the organization is several hundred engineers and the size of the projects can be between 80 and 200 engineers.² Projects are more and more often executed according to the principles of Agile software development and Lean production system referred to as *Streamline Development* (SD) within Ericsson [8]. In summary, the principles of Streamline Development postulate that software development is organized in a “factory-like” environment. The concept of factory-like development is similar to manufacturing systems – using “stations” which are responsible for a particular part of the development – e.g. estimating stations or requirements, design and implementation station, network testing. In this environment various disciplines are responsible for parts of the process: design teams (cross-functional teams responsible for complete analysis, design, implementation,

and testing of particular features of the product), network verification and integration testing, etc.

A noteworthy fact is that in SD the releases are frequent and that there is always a release-ready version of the system: referred to as *Latest System Version* [8]. It is the defects existing (and known) in that version of the system that are of interest for the project and product management. Ideally the number of known defects in that version of the system should be 0, however, during development (i.e. between releases) this number might vary as there might be defects which are discovered during the process of integrating new features (parts of code) into the Latest System Version. These defects have to be removed before the product is released to the customer – this removal is the responsibility of the *release project*, which is a project aimed at packaging the product and preparing it for the market availability.

The stakeholders for the defect backlog forecasts are project managers as it is their role to assure resources to fix the defects before the release date. Their information need in this context is: *Are we going to achieve the zero-defect goal at the release date given the resources we have?* This information need requires the information about the current defect backlog level, the forecasted trend in the defect backlog and the predicted number of defects in the backlog in the coming week or two.

It is important at this point to define the concepts which are used in the paper:

- **Defect backlog** is the set of all known and unresolved defects in the project; defect backlog is stored in the defect database. The *level of defect backlog* is then defined as the number of defects in the backlog.
- **Defect inflow** is the number of newly discovered defects reported into the defects database during a given time frame (in our case 1 week).
- **Defect outflow** is the number of defects resolved during a given time frame (in our case 1 week).
- **Prediction** is the predicted number of defects in the defect backlog in the coming week.
- **Trend** is the general tendency in the number of defects in the defect backlog over a period of 3 weeks.
- **Forecasting** is the projection of the trend and number of defects in the backlog in the coming week.

Given the above definitions and the information need of the stakeholder, we can discuss our previous attempts and why the methods like multivariate regression failed.

3.3. Defect management: why we failed before

The defects in projects at Ericsson are managed according to a dedicated lightweight process. Each defect has to go through a number of stages:

- **Submitted:** the defect is submitted by the tester or designer who found it (the submitter).
- **Assigned:** the defect is assigned to a designer who is intended to resolve the defect.
- **Resolved:** the defect has been resolved by the designer and the solution waits to be verified by the testers.
- **Verified:** the solution is verified by the testers.
- **Closed:** the defect report is closed.

These states have significant implications for our forecasts presented in the coming sections, in particular making such methods as multivariate linear regression unusable since:

- **Why we failed in Project A:** as removing defects might lead to introducing new ones and the process from submitting to clos-

¹ Due to the confidentiality agreement with Ericsson we are not able to provide the exact numbers of the organization, its products or the geographical location.

² Due to the confidentiality agreement we are not allowed to provide the exact numbers here.

ing a defect takes time, the new defect might come with a certain latency, which cannot be predicted as e.g., the latency depends on the severity of the original defect, which is not known until late in the process of resolving the defect. In the future we plan to evaluate SEVERIS for predicting the severity [27]:

- The implication is that no viable statistical model can be built for this phenomenon, especially if this is combined with the fact that this is a large project (100–200 persons).
- *Why we failed in Project B:* the verification phase make it not possible to affect the defect backlog directly by assigning more resources to resolving the defects:
 - Adding more designers to resolving defects requires adding more test resources (equipment and testers) for verification before closing; similar is true for the follow-up phase.
 - The implication is that the time required for the whole process is very hard to predict using statistical methods.
- *Why we failed with regression methods in both Projects A and B:* using variables like the number of designers who can fix defects to predict the defect outflow leads to paradoxical situations – e.g. adding new designers suddenly changes the situation from problematic to acceptable (see also the motivating example in the next chapter):
 - The practitioners' view on this can be captured in the following quote from the stakeholder: *"The only way to improve [author: decrease] the defect backlog is to fix defects, not change the project's context like the number of designers."*

This dynamic environment of a project and the strict defect management process made it very difficult to construct a 'good-enough' model for forecasting the defect backlog. The requirements from the company were that the error should be at most 20%, and in most cases below 10%. In the next section we briefly describe the attempts with such methods multivariate linear regression and analogy-based comparisons and the improvements and evolving research questions.

This situation seems to be common in the software development industry – an observation which we base on our previous collaborations with other companies.

3.4. Previous attempts

The initial objective of this research project was to improve the predictions of defect inflow³ in large software projects [39]. However, after developing prediction models based on statistical methods and evaluating them the goal has been adjusted. The final goal was to create a robust method for predicting the level of defect backlog (not inflow) with the aim to help projects to react upon the predictions before they become reality. This goal was dictated by the need to increase the quality of the product at release by acting before the problems occur.

Ericsson's project managers involved in the beginning of this research project were working with a large software project – call it Project A and had a clear requirement that the predictions should (on average) be at least 80% accurate. This meant that the average error in the predictions should not exceed 20%. The first predictions created using the multivariate linear regression, reported in [39] had average error between 28% and 375% depending on the number of predictor variables and method (see Table 1). Hence the predictions required improvements.

The research which led to these predictions was conducted over a period of several months following Project A until its finish, con-

Table 1

Extract of prediction accuracy in Project A for 1 week interval [39].

Model	Type of model	MMRE (%)
Project milestone progress	Multivariate linear regression + PCA over milestone progress [39]	52
2 weeks moving average	Moving average	34
3 weeks moving average	Moving average	38
Test progress – best statistical models	Multivariate linear regression + PCA over test progress [39]	58
Test progress – statistics and expert combined	Multivariate linear regression over test progress (variables chosen by experts) [39]	28
Expert estimations	Expert estimates based on historical data	375

stantly evaluating the accuracy of the predictions. One of the methods which we compared during the study was using *moving average* of defect inflow as a prediction method (the idea was that the predicted defect inflow is a moving average for the last 3 weeks plus a constant). The MMRE of the moving average was 38%, which was not good enough for the management of Project A, but the method significantly simpler than multivariate linear regression.

In order to improve and reach the required accuracy level we conducted a more detailed research project on a medium-size project (Project B) which lasted for approximately half-a-year which we followed from its start to finish. The research project was conducted in order to understand the factors (independent variables) used to predict the defect inflow and observe their fitness as predictions by following Project B and discussing predictions on a regular, weekly, basis. The goal was to construct more accurate prediction models and to evaluate the mathematical models w.r.t. the empirical world (entities, relationships, and empirical laws [23,40]). During this research project we used the following three methods for constructing predictions:

- *Multivariate linear regression preceded by Principal Component Analysis (PCA), referred to as statistics.* From the initial set of over 50 variables (e.g. test progress, project progress, defect inflow from previous weeks, details of this phase are reported in [39]) we chose the ones which influenced the defect inflow most (in the mathematical sense) using PCA. Using PCA resulted in identifying seven variables, which were then used to construct multivariate linear regression model for predicting the defect inflow.
- *Analogy-based prediction preceded by Principal Component Analysis referred to as analogy.* We used the same set of seven variables as identified using PCA and used analogy-based estimation with Euclidian distance function to compare weeks. The variables used for calculating similarity were (found in PCA as covering over 80% of variability):
 - Number of test cases planned in integration testing 4 weeks before the predicted week – ITCpl.
 - Number of test cases executed in integration testing 4 weeks before the predicted week – ITCex.

We used these two variables to calculate the distance between the predicted week and a set of ca. 80 weeks in the analogy database (from previous projects which we found similar to Project B).

- *Analogy-based prediction preceded by expert weighting of independent variables referred to as analogy + expert.* We asked the expert to provide us with the list of variables which he judged as variable which influences the defect backlog the most – and therefore should be considered when comparing

³ Please note the difference: predicting defect inflow and not the level of defect backlog (which seems to be more complex phenomenon as it consists of both inflow and outflow/removal of defects).

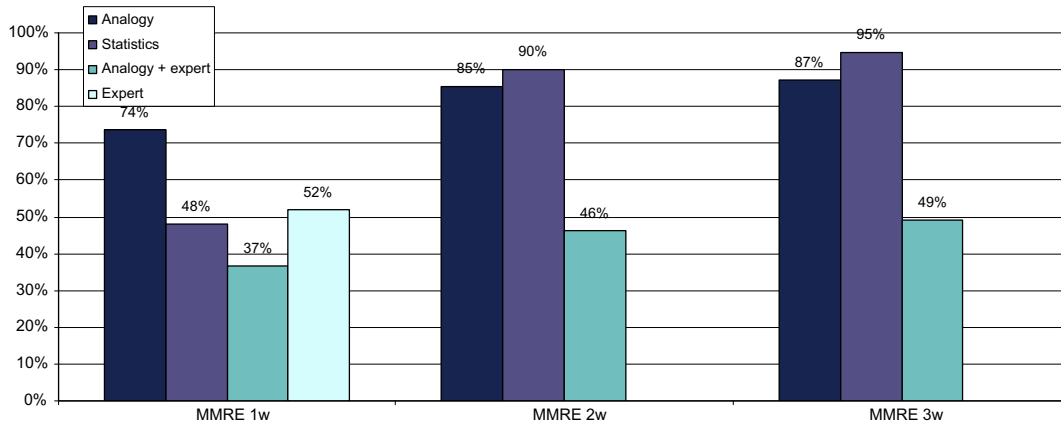


Fig. 1. MMRE for predictions in medium-size project.

weeks. He was also asked to weight that influence on the scale 0 to 1. The expert judged the following variables to be the most influential (weights in parentheses):

- Number of test cases planned for the predicted week (1.0).
- Number of integration test cases planned 1 week before the predicted week (0.75).
- Defect inflow 2 and 3 weeks before the predicted week (0.75 and 0.75 resp.).
- Number of integration test cases not executed but planned 2 and 3 weeks before the predicted week (0.5 and 0.3 resp.).
- Number of integration test cases passed 2 and 3 weeks before the predicted week (0.5 and 0.3 resp.).
- Number of integration test cases failed 2 and 3 weeks before the predicted week (both weighted 0.3).

Although after the co-linearity check we have found that there are several variables here that are correlated (which we also suspected since both the expert and the PCA algorithms started from the same set of variables) we respected the practitioners' opinion and used this in the evaluation.

- Expert estimations referred to as *expert*. Every week we asked the expert to provide us with the independent prediction of the defect inflow.

It is important to characterize the expert at this point. The expert was the project manager of Project B who had several years (over 10) of experience in working with defects in Project B and its predecessors – creating and using predictions, allocating (re-allocating) resources for removing the defects, etc. He was also formally responsible for making predictions in Project B.

As an example let us provide a list of independent variables for predictions 1 week in advance:

- Defect inflow from previous weeks (see [39,41])
 - Number of defects reported 1 week before the predicted date – D_{ii-1} .
- Test progress:
 - Number of functional test cases planned for 1 week before the predicted week – $FTTCpl_{i-1}$.
 - Number of field test cases passed for 2 weeks before the predicted week – $NTTCpsi-1$.
 - Number of field test cases planned for the predicted week – $NTTCpli$.
 - Number of system test cases failed 3 weeks before the predicted week – $STTCfi-3$.

- Accumulated number of field test cases passed for 2 weeks before the predicted week – $ANTTCpsi-2$.
- Number of functional test cases failed 1 weeks before the predicted week – $FTTCfi-1$.

The formula to predict the defect inflow in week i ($Di(i)$) was: $Di(i) = 16 + 0.4 * Di_{i-1} + 0.3 * FTTCpl_{i-1} - 0.611 * ANTTCpsi_{i-2} + 1.922 * NTTCpsi_{i-1} + 1.22 * NTTCpli - 1.1 * STTCfi_{i-3} + 0.621 * FTTCfi_{i-1}$. Although we could explain the presence of each of the variables in the formula and (to a degree) the coefficients, we still could not control whether the delicate mathematical relationships captured in the coefficients correspond to a changing empirical world. Since we could not guarantee that, using the predictions required the presence of researchers to control that.

As a result, we obtained prediction models which were less accurate than statistical prediction models for Project A. The Mean Magnitude of Relative Error (MMRE) chart in Fig. 1 presents the results. The results show that the analogy-based predictions were the best, although still not within the 20% error limit.

During the final evaluation with the stakeholder we reached the following conclusion:

- The predictions changed a lot for the same week – e.g. predicting for week 4 could render very different results when predicting 3 weeks in advance and 1 week in advance.
- In most cases the exact value of the defect inflow was not as important as the trend in the inflow related to the current backlog of defects which are not fixed. For example, it is in general problematic when the defect backlog is high and the predictions show that many new defects are expected to be reported. However, the same predicted defect inflow might not be as problematic when the defect backlog is low.

During the evaluation we identified the need for a new approach (new method) for predicting defect inflow. We observed the need to use trends in defect backlog rather than complex statistical methods for predicting the actual defect inflow per week. The stakeholder also suggested predicting the value once, not predicting (with varying accuracy) 3–2–1 weeks before the actual value. The 'instability' of these three predictions decreases the trust in the number of defects which is predicted.

We also identified the need for this forecast to be 'dynamic' – i.e. changing based on the situation in the project. For example, taking into consideration the phase of the project and the number of designers in it. In order to highlight the problem which is addressed in the research presented in this paper we provide a moti-

vating example in Section 4.1. However, we need to summarize the lessons' learned which are important for the remaining of the paper; presented in Section 4.

4. Requirements for defect backlog indicator

In this section we present the motivation, need and context of the dynamic defect backlog indicator and how it was developed. We refer to the indicator as *dynamic* as the interpretation of the indicator is based on the context of the project on a weekly basis. We explain this in this section.

4.1. Motivating example

The example presented in this section is a real-world case taken from a project at Ericsson. The values are naturally re-scaled and the time-scale is much shorter than in reality. This particular example was used when discussing the idea with stakeholders and managers in the organization during problem identification and the final action planning. In the example we consider the current number of defects, not the prediction, which is done for the sake of simplicity of the example, while it does not affect its generality. In our research we work with predicted number of defects and forecasts.

Let us consider a case when we observe the defect backlog in a small project with the metrics illustrated in Fig. 2 and Fig. 3.

In Fig. 2 the indicator for the defect level has a criteria set at the beginning of the project. This means that in week 4, the number of designers who can potentially resolve the defects is not taken into consideration. This information is taken into account in Fig. 3, where we can observe that the interpretation of the situation is different: the defect backlog is still manageable since the number of designers who can influence it is larger than in previous weeks.

The problem highlighted in this example was named the problem of *static decision criteria*, i.e. the criteria for levels problematic,

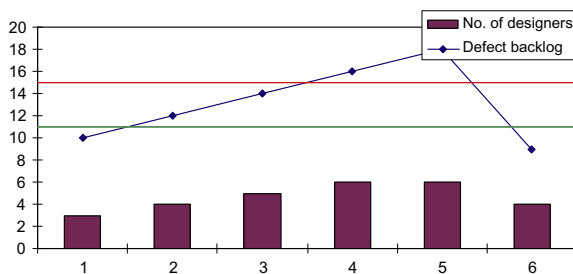


Fig. 2. Defect backlog and number of designers in an example small-size project with static decision criteria.

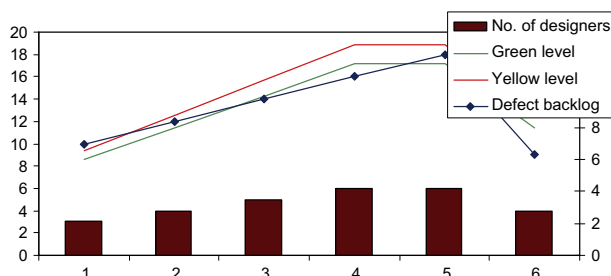


Fig. 3. Defect backlog and number of designers in an example small-size project with "dynamic" decision criteria.

warning, and acceptable are set once and do not reflect the situation in the project.

After the discussions with the stakeholder we realized that these factors must be included in the criteria. Including them in the criteria resulted in having "dynamic" decision criteria, which changed over time, as shown in Fig. 3.⁴

The formula (setting the boundary for the acceptable level of defect backlog) used in this example to set the green level (green line) is:

$$\text{green_level (week)} = \text{number_of_designers (week)} * 2 \quad (1)$$

The formula is based on the assumption that on average one designer can remove two defects per week (based on Ericsson's processes, kinds of defects, etc.). Although being a simplification, the complexity of the formula is similar to the real complexity of prediction formulas used previously at Ericsson. The formula for the red level is similar and has a constant. These formulas were the starting point for action execution in the final cycle of our action research project.

Although Eq. (1) illustrates the needs of using metrics like the number of designers to judge the value of the defect backlog, it can be manipulated. One can influence the judgment by adding the number of developers to the project without resolving any defects. For example 10 defects in the backlog can be both "green" and "red" depending on the number of designers assigned to fix them. However, the number of defects might still be too high to be accepted for the product. Since this "manipulation" can occur, the stakeholders required a model which would be reliable and robust to such manipulations.

4.2. Requirements for the forecasting model

Based on this identified need for more accurate forecasts of trends and dynamic adjustments of decision criteria, we developed a new method for forecasting the level of defect backlog. The requirement from the stakeholders at Ericsson was that the models should help the project to achieve the goal of having 0 defects in the backlog at the release date, i.e. it should support the design team to control whether they are on "on the right track", as presented in Fig. 4.⁵

In the figure we can see the indicator of the trend in defect backlog – the arrow which shows the predicted trend updated on a weekly basis. We can observe that the number of defects in the backlog is decreasing, and the situation is manageable in weeks 1–3, but it changes in week 4 and based on the predicted trend, the situation in week 5 is predicted to improve, but not sufficiently to achieve the goal of 0 defects at the release date.

The above example illustrates the need for communicating the trend in defect backlog to the stakeholder together with the predicted number of the defects in the backlog.

In addition to the above requirements we also found that the decision whether the level is "green" or "red" should be left to the stakeholder, and not built into the indicator – which was in contradiction to our previous experiences (see [10]). Despite the initial hesitation towards that (especially as it was contradictory to the previous experiences) we decided to respected the wish of the stakeholder. The stakeholder wanted to have the indicator that would show the future trend instead.

The indicator which fulfills the criteria is based on the formula presented in the next section and is presented to the stakeholders using methods shown in Section 5.2.

⁴ For interpretation of color in Figs. 1–5, the reader is referred to the web version of this article.

⁵ The line denoting the desired level is based on the discrete numbers of defects in the backlog.

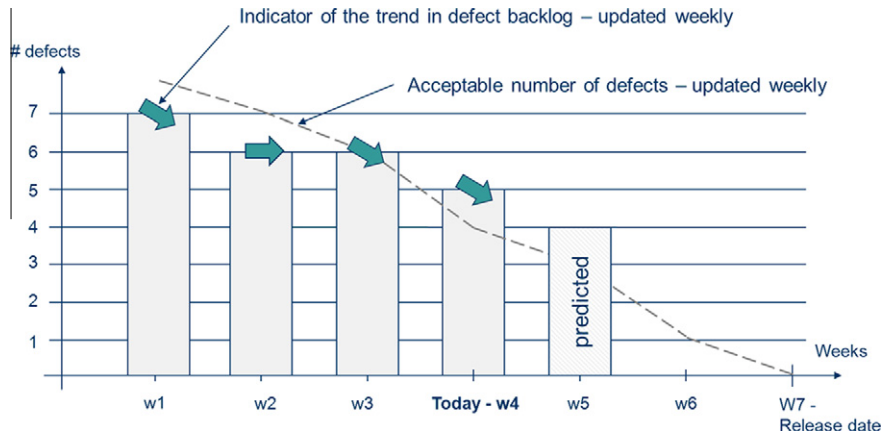


Fig. 4. A general model showing how the defect backlog indicator should “behave” in the project.

5. Forecasting defect inflow

In this section we describe the method for forecasting the defect inflow which is the main contribution of our work: the formula for predictions, the indicator, and its presentation to the stakeholder. This section corresponds to the action execution in the final cycle of our action research project.

5.1. Formula for predicting the number of defects in the backlog

The method, which consists of a formula for predicting the number of defects in the backlog, an indicator with decision criteria, and a method for presenting the information to the stakeholder, was developed together with experienced integration leaders, project leaders, and quality managers from Ericsson.

The formula (formula for predicting defect backlog using moving averages) used to predict the level of defect backlog is:

$$db(i) = db(i-1) + \frac{di(i-1) + di(i-2) + di(i-3)}{3} - \frac{do(i-1) + do(i-2) + do(i-3)}{3} \quad (2)$$

where $db(x)$ is the defect backlog in week x , $di(x)$ is the defect inflow in week x (i.e. the number of defects reported during week x), the same variable as presented in Section 3.4 and [39], $do(x)$ is the defect outflow in week x (i.e. the number of defects removed during week x).

This formula captures the following empirical relationship: the predicted number of defects in the backlog next week is the sum of the actual number of defects during this week with a predicted defect inflow and predicted defect outflow. The predicted defect inflow and outflow are the moving averages of the last 3 weeks. During the previous studies in the large project (described in section “previous attempts”) we found that the moving average of defect inflow is a good estimator of the defect inflow with MMRE of 30%. The simplicity of using the moving average outweighed the accuracy of multivariate linear regression models (MMRE of 24% with seven variables used in the regression equation [39]).

Eq. (2) was used to develop the decision criteria for the dynamic indicator – forecast for the defect inflow, described in Section 4.2.

5.2. Dynamic defect backlog indicator

The formula for predicting the defect inflow was used to create an indicator for the defect backlog. The indicator is the arrow for

showing whether the defect backlog is forecasted to increase or decrease in the coming week:

- Pointing up if the forecast shows that the defect backlog will increase in the coming week – i.e. if $db(x) < db(x+1)$.
- Pointing down if the forecast shows that the defect backlog will decrease in the coming week – i.e. if $db(x) > db(x+1)$.
- Pointing right if the forecast shows that the defect backlog will remain the same – i.e. $db(x) = db(x+1)$.

Fig. 5 shows the graphical presentation of the indicator as MS Windows Vista gadget with the indicator in the center.

The gadget contains the most important information for the stakeholder:

- Date when the prediction was created.
- Validity information – whether all calculations were correct and the data was up-to-date (green – no problems with validity, red – problems with validity the stakeholder should not rely on the data). We use an automated information quality assessment tools for notifying the stakeholder whether he can trust the predictions or not [42].
- Current defect backlog – the number of defects in the backlog on the date of prediction.
- Predicted defect backlog – the number of defects predicted for Monday on next week.
- Current week – the number of the current week in the format wYWW (Y = year, WW = week).
- Predicted week – the number of the predicted week in the same format.

The gadget itself is a way of presenting the information to the stakeholder without requiring the stakeholder to manually open files or websites to check the information. All calculations are done in by the measurement system implemented as an MS Excel file based on our framework [10]. The measurement system fetches

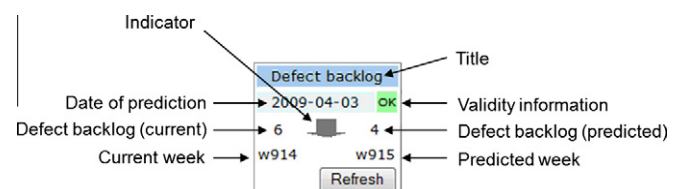


Fig. 5. MS Vista Gadget presenting the defect backlog indicator.

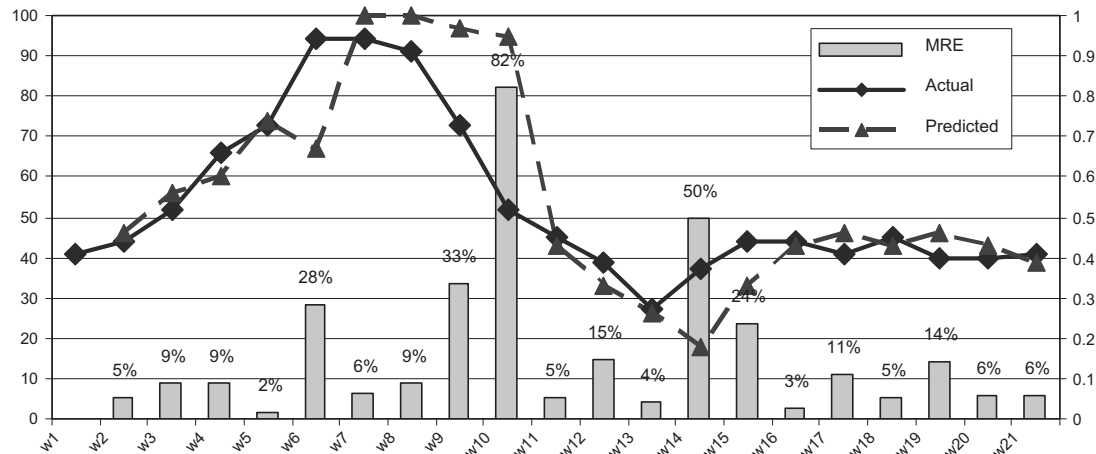


Fig. 6. Predicted and actual defect backlog with associated prediction error (MRE) per week.

the data used for predictions from databases, makes the calculations, and creates the gadget with the information.

6. Evaluation

We evaluated our method for forecasting defect backlog as part of the action research project conducted at Ericsson. The identification of the problem and the need for this new method was described in section “previous attempts”. In this section we describe the set-up of a measurement system with the dynamic defect backlog indicator, we briefly describe the project, where it was used, present a summary of the log from weekly evaluation meetings with the stakeholder, show a graph with the predicted vs. actual defect backlog, and finally compare the MMRE of the predictions to the MMRE from our previous attempts. This section corresponds to the action evaluation in the final cycle of our action research project.

6.1. Context of the evaluation

The project that was chosen for the evaluation was executed according to the Streamline principles [8] and was developing the software part of one of products in the mobile telephony network. This particular project size was ca. 100 persons. The evaluation started when this project adopted the streamline principle and lasted until the release of the new version of the product.

This product was chosen based on the worst-case scenario principle: the project was a new release of the product which historically was characterized as very complex (due to dependencies on other products, protocols and its placement in the mobile telephony network) and rather unpredictable. The middle-size project used in our previous attempts was a sub-project in one of the predecessors of the current project and was also characterized by high degree of unpredictability. Our claims are that if the indicator is useful in the case of this project, then it will be even more useful in more stable projects. By that we mean even more accurate predictions and predicting longer in advance.

6.2. Method

During the evaluation period we set-up a measurement system which was available on the intranet and the MS Vista Gadget for the stakeholder. Our intention was to work as closely as possible with the stakeholder without introducing disturbances to his work. The stakeholder was the integration leader responsible for moni-

toring and acting upon the defect backlog level. He was an experienced engineer with over 10 years of experience as project manager, and integration leader. The stakeholder used the indicator to monitor the defect backlog and to spread the information to the project members. Apart from the informal discussions, we met the stakeholder every week for a formal 15-min evaluation of the indicator. During that evaluation we mainly asked the following question:

1. Did the indicator show the correct trend of the defect backlog (i.e. correct w.r.t. actual values)?

We also asked the stakeholder about the situation in the project in order to capture the relationship between the mathematics (the metrics, the indicator and the predictions) and the empirical world (situation in the project).

A quality manager was also involved in this project through informal discussions (several times per week) during the project and an evaluation after the project. The quality manager has over 6 years of experience in his role (taking part in large software projects), as well as in the role of project manager (for sub-projects).

For the accuracy evaluation we use the Mean Magnitude of Relative Error (MMRE, [43]) as the metric: the lower the MMRE, the better the predictions.

6.3. Results from evaluation

Although the accuracy of the predictions of the exact number of defects in the project was not important (but the trend), we show the accuracy of the prediction as it shows how good the prediction formula itself is. The accuracy of the trend (the indicator) cannot be measured using MMRE as the indicator is the “arrow”. The predicted and actual defect backlog with the associated Magnitude of Relative Error (MRE) is presented in Fig. 6. The graph was constructed week-by-week, i.e. not at the end of the project with historical data. The graph is re-scaled so that the largest number in the defect backlog (predicted value for week w7) is 100. MRE has not been re-scaled.

The chart shows that there are two periods of time when the predictions were incorrect: weeks w8–w10 and w14–w15.⁶ The situation was explained by the fact that the project took measures in w8–w10 in order to decrease the defect backlog. Significantly

⁶ Week w6 was removed from the analysis since it was an incorrect data point – there was a technical problem with the measurement system during that vacation week.

more resources were assigned to removing the defects, re testing the products and closing the defects. In week w14, these resources were no longer available.

During the first 5 weeks the stakeholder was rather skeptical for the forecast model, usually claiming that:

“It is rather not probable that we will have <xx> defects in the backlog next week, because ...”

The justification was usually that the situation in the project did not point in the same direction as the indicator. However, after the period of first 5 weeks, his attitude changed given the fact that the forecasts came true and the indicator showed the right trend.

When asked why the project took measures in week w10, the stakeholder pointed out that the situation is not acceptable and that the forecast show that it is going to get even worse; hence they needed to take measures to avoid this. When asked about the situation in week w12, whether the indicator contributed to the project taking actions the response from the stakeholder was:

“The indicator helped us by creating «crisis awareness» and thus motivated us to take actions to avoid problems”.

In week w14 the project celebrated the lowest number of defects in the backlog for a number of releases in this product (previous projects). The level of defect backlog for the weeks w15–w21 was said to be an acceptable level in the project. The reason for accepting a non-zero number of defect was the take-over by the release project. The release projects are projects which take the Latest System Version and perform the necessary packaging of the product and resolve the remaining defects (usually minor defects like inconsistencies in user documentation, spelling mistakes).

The dynamic TR backlog indicator was adopted in the organization and used continuously after concluding our research activities (and is still used at the moment of writing of this paper – ca. half-a-year later). The reason why this indicator caused the project at Ericsson to take measures was the fact that the stakeholder was involved in the development of the forecast model (the formula) and the indicator was spread in the project. The project management trusted the indicator (as it showed the correct trends for the first 10 weeks). An important aspect in this project was that, in the contrary to the previous attempts, we worked closely with the stakeholder and used his experience to develop the formula. In addition to that we used mathematical models.

The quality manager pointed out during the final evaluation that the simplicity of the formula used for prediction was one of the key elements of the success. The previous attempts resulted in complicated mathematical formulas which were hard to adjust when the project changed – e.g. when testing was performed in another way [39]. The formula used in this project was well understood by the organization and showed that the only way to improve w.r.t. defect backlog is to remove/repair the defects, not just re-assign resources. This opinion was also shared by the stakeholder. The quality manager pointed out that the empirical focus of this research project was another important factor:

“You can use mathematics, but then there is the empirical world, which is not based on mathematics.”

An important aspect of our evaluation is to observe the value of the indicator during the project. In Table 2 we present the predicted and the actual trends for each week – column 2 denotes the direction of the arrow (e.g. “Up” when the trend was predicted to increase) and column 3 shows whether the trend raised or fell.

In 10 times out of 20 the trend predicted was the same as the actual trend, whereas in 10 weeks it was different from the actual trend. During the end of the project the trends in the defect backlog were shifting up-and-down and these shifts made it hard to predict

Table 2

Comparison of predicted and actual trends including MMRE of predictions.

Week	Predicted trend	Actual trend	MMRE of predictions (%)
W1	N/A	N/A	N/A
W2	Up	Up	5
W3	Up	Up	9
W4	Up	Up	9
W5	Up	Up	2
W6			–See footnote on prev. page
W7	Up	Up	6
W8	Same	Down	9
W9	Up	Down	33
W10	Up	Down	82
W11	Down	Down	5
W12	Down	Down	15
W13	Down	Down	4
W14	Down	Up	50
W15	Down	Up	24
W16	Down	Up	3
W17	Up	Down	11
W18	Up	Up	5
W19	Same	Down	14
W20	Up	Same	6
W21	Same	Up	6

the actual trend. However, if we analyze the MMRE for the same weeks – week 16–21 – then one can see that the predicted values were not very far from the actual values.

In this situation we have found that the stakeholder appreciated the information about the predicted value of the number of defects in the backlog and the indicator for the forecasted trend. Having this information helped him to make better decisions about how to handle the defect backlog.

The final part of our evaluation is the comparison of prediction accuracy with our previous attempts.

6.4. Comparison to previous attempts

In this section we present a short comparison of MMRE of the predictions used in our previous attempts (described in Section 3.4) and the forecasting formula. The summary of MMRE is presented in Table 3.

The table shows that the forecasting model, i.e. our prediction formula (Eq. (2)) presented in this paper, has the best accuracy. Although we considered the comparison to existing, state-of-the-art methods (w.r.t. MMRE) we did not perform that since we cannot really control whether such a comparison would be meaningful. The main reason for that is the fact that in the published papers it is not visible if the projects, where the predictions are applied, can be meaningfully compared.

Table 3

Comparison of MMRE for our previous studies and the forecasting model.

Study	Method	MMRE (%)
Project A (large)	Multivariate linear regression + PCA over milestone progress [39]	52
	Multivariate linear regression + PCA over test progress [39]	58
	Multivariate linear regression over test progress (variables chosen by experts) [39]	24
Project B (medium)	Analogy-based estimation over all available variables	74
	Multivariate linear regression + PCA over test progress	48
	Analogy-based estimation over variables chosen by expert	37
	Expert estimates	52
Streamline	Forecasting	16

It is also important to note at this point that our method is best suited for projects which are executed according to the principles of lean software development. The characteristics of these projects (see Section 3.1) make the moving average a simple, yet sufficiently robust method. In particular frequent releases, constant availability of Latest System Version and the need to keep the defect backlog low at all times justify the use of moving average. The latency between making an attempt to fix a defect and its actual resolution make the 3 week period a “window” for the moving average.

There exist evidence of moving average to be a very poor prediction method, but the context and henceforth the underlying empirical relationships are different from our case. For example, Li et al. [44] have reported on the insufficiency of moving averages for predicting the inflow of field-defects. The difference between their result and ours can be explained by the fact that field-defect profiles usually have a distribution close to the Weibull distribution (and are expected to have such a distribution based on the reliability theory – e.g. the Rayleigh model). In Lean software development the defect inflow profile should be as low as possible and because it is a continuous development process (before release), Weibull distribution and the Rayleigh model do not apply here.

7. Threats to validity discussion

In this section we briefly discuss the threats to validity of our results, in particular the evaluation of the new method. The threats are grouped according to categories advocated by Wohlin et al. [45].

The main threat to the *external validity* of the results is the action research approach in our work. As the formula, the indicator, and its presentation were created together with Ericsson and for Ericsson, there is a danger that the results are not generalizable to other companies. Although it is a threat, we claim that if a company uses a dedicated process for defect management, which is stable over projects, then the method will be applicable. This claim is supported by the fact that the formula (Eq. (2)) for predicting the defect backlog uses the data from the same company/project for predicting and thus is not biased by Ericsson's context. The fact that it worked at Ericsson provides evidence that it is indeed applicable and provides satisfactory results in an industrial context.

The main *construct validity* threat is the use of MMRE as the main metric for accuracy. We decided to limit ourselves to only this and instead present the re-scaled actual values for predictions. We decided to let the engineers and managers applying our method in other companies to judge for themselves whether such accuracy fulfills their expectations. Another important threat in this category is the limited number of interviews – the stakeholder and the quality manager. We decided to limit ourselves only to decision makers w.r.t. defect backlog as it is them who are the main audience for the indicator of defect backlog. This decision was caused by the fact that we intended to observe how the indicator helps the organization to improve, which made the decision makers the most suitable group.

The main *internal validity* threat is the fact that the study was done over a long period of time and therefore there is a risk of the learning effect. In other words, the improvement of the defect backlog observed in the study might have been caused by organization's/stakeholder's learning process in parallel to the indicator and not the presence of the defect backlog indicator. Even though we cannot completely rule out such a factor, the claims of the stakeholder that the indicator contributed to the improvement of the defect backlog by creating crisis awareness show that the indicator indeed played an important part in the improvement. This

means that even if learning took place, it was a result of the indicator and not vice versa, which causes no causality problem. Another threat which we see is the choice of the case project. There is a danger that choosing other project would render different results. Our choice, however, was dictated by the worst-case scenario principle. The project which we used in the evaluation was a complex one which historically (i.e. previous projects on the same product) was characterized by unpredictability of defect backlog and very dynamic changes to the project. There is naturally an important confounding factor in the study – the predictions being used at the same time when being evaluated. The project taking actions to avoid problems decreased the accuracy of the predictions. However, we see this as a positive effect, as the goal was to improve the situation at the company, not to have exact (but negative) predictions – i.e. add value to the company.

The main *conclusion validity* threat is the fact that we conclude the usefulness of the backlog indicator in improving the defect backlog. Despite the lack of statistical data to support this claim, the statements from the stakeholder on the usefulness of the indicator as well as the statement from the quality manager that working closely with the stakeholder resulted in much better predictions than before, we claim that there is a relationship between the improvement of the defect backlog and the use of the indicator.

8. Conclusions

In large software projects executed according to Streamline Development, which is a mix of Agile and Lean development principles, managing defects is one of the main focuses of the project management team. The project management needs to know whether the goal of 0 known defects can be achieved at the release data and therefore the predictions of the defect backlog (the set of all known defects in the developed product) and the forecasted trend (whether it is going to raise or fall) become even more important than in traditional projects.

In this paper we presented a new way of predicting the level of defect backlog in large software projects. We predict a value of the defect backlog, while we communicate the forecast of a trend to the stakeholder. The results presented in the paper are an outcome of an almost 3-year long action research project which included evaluation of several prediction methods: multivariate linear regression, analogy-based estimations, expert estimates. As a result of these evaluations and the long term commitment from the industrial side we were able to develop a relatively simple formula for predicting the level of defect backlog in the project which was characterized by high accuracy (Mean Magnitude of Relative Error of 16%).

One of the most important side-findings in this study, which was not aimed at from the beginning is the perspective of the practitioners on the problem of predicting defect backlogs. We have found that the simplicity of our model is of a crucial importance for the stakeholders in the company. More complex defect prediction models are perhaps more accurate, but they are also very complex. This complexity of the methods makes them very vulnerable to changes in the ways of working at companies (a very common situation in software industry), which means that stakeholders are reluctant to adopt them.

During the evaluation of this new method in a project at Ericsson, it was found that the indicator supported the project management and initiated measures to decrease the level of the defect backlog in the project. The method was continued to be used after our research project concluded and is now being spread to other projects. The simplicity of the method and straightforward presentation make it easy to adopt and do not require significant amount of effort.

In our future work we intend to evaluate the method in other, more traditional projects, and port it to other companies in other domains, e.g. automotive, in our other research projects.

Acknowledgements

The authors would like to thank Ericsson AB for the received support; in particular the managers supporting us and the Software Architecture Quality Center.

References

- [1] P. Tran, R. Galka, On incremental delivery with functionality, in: Tenth Annual International Conference on Computers and Communications, Scottsdale, AZ, USA, 1991.
- [2] D. Graham, Incremental development and delivery for large software systems, in: IEE Colloquium on Software Prototyping and Evolutionary Development, London, UK, 1992.
- [3] A. Cockburn, *Agile Software Development*, Addison-Wesley, Boston, MA, USA, 2002.
- [4] K. Schwaber, M. Beedle, *Agile Software Development with Scrum*, Prentice Hall, Upper Saddle River, NJ, USA, 2002.
- [5] K. Beck, C. Andres, *Extreme Programming Explained: Embrace Change*, Safari Tech Books Onliner, Boston, MA, USA, 2005.
- [6] M. Poppendieck, T. Poppendieck, *Implementing Lean Software Development: From Concept to Cash*, Addison-Wesley, Boston, MA, USA, 2007.
- [7] J. Womack, D. Jones, D. Ross, *The Machine that Changed the World: Based on the Massachusetts Institute of Technology 5-Million-Dollar 5-Year Study on the Future of the Automobile*, Rawson Associates, New York, NY, USA, 1990.
- [8] P. Tomaszewski, P. Berander, L.-O. Damm, From traditional to streamline development – opportunities and challenges, *Software Process Improvement and Practice* 2007 (1) (2007) 1–20.
- [9] International Standard Organization and International Electrotechnical Commission, ISO/IEC 15939 Software engineering – Software measurement process, International Standard Organization/International Electrotechnical Commission, Geneva, 2007.
- [10] M. Staron, W. Meding, C. Nilsson, A framework for developing measurement systems and its industrial evaluation, *Information and Software Technology* 51 (4) (2008) 721–737.
- [11] O. Adelakun, Quality – what does it really mean for strategic information systems, in: Second Conference on Information Quality, Turku, Finland, 1997.
- [12] T. Ball, N. Nagappan, Static analysis tools as early indicators of pre-release defect density, in: Twenty-Seventh International Conference on Software Engineering, IEEE, St. Louis, MO, USA, 2005.
- [13] A.M. Neufelder, How to predict software defect density during proposal phase, in: National Aerospace and Electronics Conference, Dayton, OH, USA, 2000.
- [14] Y.K. Malaiya, J. Denton, Module size distribution and defect density, in: Eleventh International Symposium on Software Reliability Engineering, San Jose, CA, USA, 2000.
- [15] P. Mohagheghi et al., An empirical study of software reuse vs. defect-density and stability, in: Twenty-Sixth International Conference on Software Engineering, IEEE, 2004.
- [16] W.W. Agresti, W.M. Evancho, Projecting software defects from analyzing Ada designs, *IEEE Transactions on Software Engineering* 18 (11) (1992) 988–997.
- [17] T. Ball, N. Nagappan, Use of relative code churn measures to predict system defect density, in: Twenty-Seventh International Conference on Software Engineering, St. Louis, MO, USA, 2005.
- [18] N.E. Fenton, M. Neil, A critique of software defect prediction models, *IEEE Transactions on Software Engineering* 25 (5) (1999) 675–689.
- [19] A. Mockus, D.M. Weiss, Predicting risk of software changes, *Bell Labs Technical Journal* 5 (2) (2000) 169–180.
- [20] J.P. Cavano, Software reliability measurement: prediction, estimation, and assessment, *Journal of Systems and Software* 4 (4) (1984) 269–275.
- [21] B. Littlewood, N.E. Fenton, City University Centre for Software Reliability, *Software Reliability and Metrics*, vol. xii, Elsevier Applied Science, London, 1991. p. 235 s.
- [22] J.H. Bailey, R.A. Kowalski, Reliability-growth analysis for an Ada-coding process, in: Annual Reliability and Maintainability Symposium, 1992.
- [23] N.E. Fenton, S.L. Pfleeger, *Software Metrics: A Rigorous and Practical Approach*, vol. XII, second ed., International Thomson Computer Press, London, 1996. p. 638 s.
- [24] M. Staron, W. Meding, Defect inflow prediction in large software projects, *e-Informatica Software Engineering Journal* 4 (1) (2010) 1–23.
- [25] L.M. Laird, M.C. Brennan, *Software Measurement and Estimation: A Practical Approach*, vol. xvi, John Wiley & Sons, Hoboken, NJ, 2006. 257 p.
- [26] P.L. Li, et al., Empirical evaluation of defect projection models for widely deployed production software systems, in: *Foundation of Software Engineering*, IEEE Computer Society, 2004, pp. 263–272.
- [27] T. Menzies, A. Marcus, Automated severity assessment of software defect reports, in: *International Conference on Software Maintenance*, IEEE Computer Society, 2008, pp. 346–355.
- [28] P. Hooimeijer, W. Weimer, Modeling bug report quality, in: *Automated Software Engineering*, Association of Computing Machinery (ACM), 2007, pp. 34–43.
- [29] B. Clark, Eight secrets of software measurement, *IEEE Software* 19 (5) (2002).
- [30] T. Kilpi, Implementing a Software Metrics Program at Nokia, *IEEE Software* 18 (6) (2001) 72–77.
- [31] C.A. Dekkers, P.A. McQuaid, The dangers of using software metrics to (mis) manage, *IT Professional* 4 (2) (2002) 24–30.
- [32] S.L. Pfleeger, et al., Status Report on Software Measurement, in: *IEEE Software*, 1997, pp. 33–34.
- [33] A. Bröckers, C. Differding, G. Threin, The role of software process modelling in planning industrial measurement programs, in: *Metrics*, IEEE, 1996.
- [34] L. Chirinos, F. Losavio, J. Boegh, Characterizing a data model for software measurement, *Journal of Systems and Software* 74 (2) (2005) 207–226.
- [35] R. van Solingen, E. Berghout, The goal/question/metric method, *A Practical Guide for Quality Improvement of Software Development*, McGraw-Hill, London, 1999. p. 195.
- [36] J. Schalken, H. van Vliet, Measuring where it matters: Determining starting points for metrics collection, *Journal of Systems and Software* 81 (5) (2008) 603–615.
- [37] R.L. Baskerville, A.T. Wood-Harper, A critical perspective on action research as a method for information systems research, *Journal of Information Technology* 1996 (11) (1996) 235–246.
- [38] G.I. Susman, R.D. Evered, An assessment of the scientific merits of action research, *Administrative Science Quarterly* 1978 (23) (1978) 582–603.
- [39] M. Staron, W. Meding, Predicting weekly defect inflow in large software projects based on project planning and test status, *Information and Software Technology*, 2007 (available online).
- [40] H. Zuse, *A framework of software measurement*, vol. xxix, Walter de Gruyter, Berlin; New York, 1998. 755 p.
- [41] M. Staron, W. Meding, Short-term defect inflow prediction in large software project – an initial evaluation, in: *International Conference on Empirical Assessment in Software Engineering (EASE)*, British Computer Society, Keele, UK, 2007.
- [42] M. Staron, W. Meding, Ensuring reliability of information provided by measurement systems, in: *Software Process and Product Measurement*, Springer, 2009, pp. 1–16.
- [43] B.A. Kitchenham et al., What accuracy statistics really measure (software estimation), *IEE Proceedings – Software* 148 (3) (2001) 81–85.
- [44] P.L. Li et al., Experiences and results from initiating field defect prediction and product test prioritization efforts at ABB Inc, in: *Proceedings of the 28th International Conference on Software Engineering*, ACM, Shanghai, China, 2006.
- [45] C. Wohlin et al., *Experimentation in Software Engineering: An Introduction*, Kluwer Academic Publisher, Boston, MA, 2000.